

DL Assignment-3

Q1) Explain working of Backpropagation algorithm.

A)

Backpropagation is the algorithm that tells a neural network how to change each weight and bias so the network's output gets closer to the target. It does this by computing derivatives of the loss with respect to each parameter efficiently using the chain rule, working backwards from the output layer to the input layer.

1) Notation & forward pass

Consider an L -layer feedforward network. For layer l :

- $a^{(l)}$ = activations (column vector) of layer l
- $z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}$ (pre-activation)
- $a^{(l)} = \sigma(z^{(l)})$ (elementwise activation function)
- Input: $a^{(0)} = x$. Output: $a^{(L)}$.

Loss (single example): $\mathcal{L}(a^{(L)}, y)$ (e.g. MSE, cross-entropy).

Forward pass: compute $z^{(1)}, a^{(1)}, \dots, z^{(L)}, a^{(L)}$ and then the loss.

2) What we need: gradients of loss w.r.t. parameters

We want

$$\frac{\partial \mathcal{L}}{\partial W^{(l)}}, \quad \frac{\partial \mathcal{L}}{\partial b^{(l)}}$$

for every layer l . Use the chain rule — compute a local error signal for each layer and reuse it.

Define the *error* (δ) at layer l :

$$\delta^{(l)} \equiv \frac{\partial \mathcal{L}}{\partial z^{(l)}}$$

(derivative of loss wrt the pre-activation). Once we have $\delta^{(l)}$:

- $\frac{\partial \mathcal{L}}{\partial W^{(l)}} = \delta^{(l)} (a^{(l-1)})^{\text{top}}$

- $\frac{\partial \mathcal{L}}{\partial b^{(l)}} = \delta^{(l)}$ (sum across batch examples if batching)

So the main problem is computing each $\delta^{(l)}$.

3) Backprop recursion (derivation)

At the output layer L :

$$\delta^{(L)} = \frac{\partial \mathcal{L}}{\partial a^{(L)}} \odot \sigma^{(L)'}(z^{(L)})$$

(where $\odot$ is elementwise product).

For earlier layers l (moving backwards):

$$\delta^{(l)} = \left(W^{(l+1)\top} \delta^{(l+1)} \right) \odot \sigma^{(l)'}(z^{(l)}).$$

This follows directly from the chain rule: $z^{(l+1)}$ depends on $a^{(l)}$, $a^{(l)}$ depends on $z^{(l)}$, etc.

Special useful simplifications:

- For *softmax + cross-entropy* the output delta simplifies to $\delta^{(L)} = a^{(L)} - y$ (very common and numerically stable).
- For *MSE (mean squared error)* and sigmoid output: $\delta^{(L)} = (a^{(L)} - y) \odot \sigma'(z^{(L)})$.

4) Vectorized mini-batch form (shapes)

For a mini-batch of size m we work with matrices:

- $A^{(l)}$ is (size of layer l) $\times$ m
- $W^{(l)}$ is (size of layer l) $\times$ (size of layer $l-1$)
- $B^{(l)}$ is (size of layer l) $\times$ 1 (broadcast across m examples)
- $\Delta^{(l)}$ (for the batch) has same shape as $Z^{(l)}$

Vectorized gradients (averaging across the batch):

$$\frac{\partial \mathcal{L}}{\partial W^{(l)}} = \frac{1}{m} \Delta^{(l)} A^{(l-1)\top}, \quad \frac{\partial \mathcal{L}}{\partial b^{(l)}} = \frac{1}{m} \sum_{i=1}^m \Delta^{(l)}_{:,i}.$$

Backprop recursion in matrix form identical but uses matrix multiplication instead of sums for each example.

5) Training step (algorithm / pseudocode)

for epoch in 1..N_epochs:

for each mini-batch (X_batch, Y_batch):

forward pass

$A[0] = X_batch$

for $l = 1..L$:

$Z[l] = W[l] @ A[l-1] + b[l]$ # (matrix multiply + broadcast)

$A[l] = \text{sigma}_l(Z[l])$

compute loss, e.g. cross-entropy or MSE

backward pass

$\Delta[L] = d\text{Loss}_{dA}(A[L], Y_batch) * \text{sigma}_{L_prime}(Z[L])$

for $l = L-1 \dots 1$ (downwards):

$\Delta[l] = (W[l+1].T @ \Delta[l+1]) * \text{sigma}_{l_prime}(Z[l])$

gradients

$dW[l] = (1/m) * \Delta[l] @ A[l-1].T$

$db[l] = (1/m) * \text{sum_columns}(\Delta[l])$

parameter update (SGD)

$W[l] = W[l] - \text{learning_rate} * dW[l]$

$b[l] = b[l] - \text{learning_rate} * db[l]$

Replace SGD updates with Adam/Momentum/etc. as desired.

6) Small numeric worked example (step-by-step)

One hidden layer network (2 inputs $\rightarrow$ 2 hidden units $\rightarrow$ 1 output). Sigmoid activations, MSE loss. Single training example.

Given

- Input $x = [0.5, -0.5]^T$

- Hidden weights $W^{(1)} = \begin{bmatrix} 0.1 & -0.2 \\ 0.4 & 0.2 \end{bmatrix}$, $b^{(1)} = [0, 0]^T$
- Output weights $W^{(2)} = [0.3, -0.1]$ (1×2), $b^{(2)} = 0.05$
- Target $y = 0.7$
- Activation $\sigma(s) = \frac{1}{1 + e^{-s}}$.

Forward pass

1. Pre-activations for hidden layer:
 - $z^{(1)}_1 = 0.1 \cdot 0.5 + (-0.2) \cdot (-0.5) = 0.05 + 0.10 = 0.15$
 - $z^{(1)}_2 = 0.4 \cdot 0.5 + 0.2 \cdot (-0.5) = 0.20 - 0.10 = 0.10$
So $z^{(1)} = [0.15, 0.10]^T$.
2. Activations (sigmoid):
 - $a^{(1)}_1 = \sigma(0.15) \approx 0.53742985$
 - $a^{(1)}_2 = \sigma(0.10) \approx 0.52497919$
So $a^{(1)} \approx [0.53743, 0.52498]^T$.
3. Output pre-activation:
 - $z^{(2)} = 0.3 \cdot 0.53742985 + (-0.1) \cdot 0.52497919 + 0.05 \approx 0.15873103$
4. Output activation:
 - $a^{(2)} = \sigma(0.15873103) \approx 0.53959965$.
5. Loss (MSE, half squared error):
 - $\mathcal{L} = \frac{1}{2} (a^{(2)} - y)^2 = \frac{1}{2} (0.53959965 - 0.7)^2 \approx 0.0128641363$.

Backward pass

1. Output delta (MSE + sigmoid):
 - $\delta^{(2)} = (a^{(2)} - y) \cdot \sigma'(z^{(2)})$.
 - $\sigma'(z^{(2)}) = a^{(2)}(1 - a^{(2)}) \approx 0.53959965(1 - 0.53959965) \approx 0.2485737$.
 - $a^{(2)} - y = 0.53959965 - 0.7 = -0.16040035$.
 - So $\delta^{(2)} \approx -0.16040035 \cdot 0.2485737 \approx -0.03984856$.
2. Gradients for output layer:

- $\frac{\partial \text{mathcal{L}}}{\partial W^{(2)}} = \delta^{(2)}, (a^{(1)})^{\text{top}} = -0.03984856 \times [0.53742985, 0.52497919] \approx [-0.0214158, -0.02091966]$.
- $\frac{\partial \text{mathcal{L}}}{\partial b^{(2)}} = \delta^{(2)} \approx -0.03984856$.

3. Backprop to hidden layer:

- Compute $(W^{(2)})^{\text{top}} \delta^{(2)} = [0.3, -0.1]^{\text{top}} \cdot (-0.03984856) \approx [-0.01195457, 0.003984856]$.
- Multiply elementwise by $\sigma'(z^{(1)})$:
 - $\sigma'(0.15) = a^{(1)}_1(1-a^{(1)}_1) \approx 0.53742985(0.46257015) \approx 0.2489863$.
 - $\sigma'(0.10) = 0.52497919(0.47502081) \approx 0.2499990$.
 - So $\delta^{(1)} \approx [-0.01195457 \cdot 0.2489863, 0.003984856 \cdot 0.2499990] \approx [-0.00297189, 0.00099373]$.

4. Gradients for hidden layer:

$$\begin{bmatrix} -0.00297189 \\ 0.00099373 \end{bmatrix} \begin{bmatrix} 0.5 & -0.5 \end{bmatrix} = \begin{bmatrix} -0.00148595 & 0.00148595 \\ 0.00049686 & -0.00049686 \end{bmatrix}.$$

So we computed the forward pass, the local errors, and the gradients for all weights and biases. Those gradients would then be used in an update step (e.g. $W \leftarrow W - \eta \frac{\partial \text{mathcal{L}}}{\partial W}$).

7) Why backpropagation is efficient

Naively computing $\frac{\partial \text{mathcal{L}}}{\partial W}$ for each weight by re-evaluating partial derivatives would be exponentially expensive. Backprop computes each needed partial once and reuses results: it propagates local gradients backward so each parameter uses local information and the propagated error. In a feedforward network the cost of backprop is about a constant multiple of the forward pass cost (same order as computing activations) — i.e., linear in the number of parameters per example.

8) Practical issues & best practices

- **Activation choice:** Sigmoid/tanh can cause vanishing gradients in deep nets. ReLU (and variants) mitigate that.
- **Initialization:** Use Xavier/He initialization depending on activation to keep forward/backward signal magnitudes stable.

- **Learning rate & optimizers:** Use learning rate schedules and adaptive optimizers (Adam, RMSProp) or SGD+momentum.
- **Batching:** Mini-batches (e.g. 32–512) trade gradient noise vs compute efficiency.
- **Regularization:** Weight decay (L2), dropout, data augmentation to prevent overfitting.
- **Gradient clipping:** Useful to prevent exploding gradients (clip by norm).
- **Batch normalization / residual connections:** Help training very deep nets (stabilize activations, gradients).
- **Gradient checking:** Numerically approximate gradients to verify implementation (finite differences) during debugging:

$$\frac{\partial \mathcal{L}}{\partial \theta_i} \approx \frac{\mathcal{L}(\theta + \epsilon e_i) - \mathcal{L}(\theta - \epsilon e_i)}{2\epsilon}$$

9) Common mistakes & diagnostics

- Too large learning rate → divergence/exploding gradients.
- Wrong activation derivative or shape mismatch → wrong gradient shapes.
- Forgetting to average gradients over the batch → steps scale incorrectly with batch size.
- Using sigmoid or tanh everywhere for deep nets → vanishing gradients.
- Not initializing biases/weights appropriately.

10) Complexity & memory

- Time complexity per pass $\sim O(\text{number of weights})$ per example (forward + backward $\approx$ constant factor).
- Memory: must store activations $a^{(l)}$ and pre-activations $z^{(l)}$ for every layer during forward pass so backward pass can use them (this is why deep networks + large batches can be memory-heavy).

Q2) What is an Artificial neural network (ANN)? Describe its basic structure and components.

A)

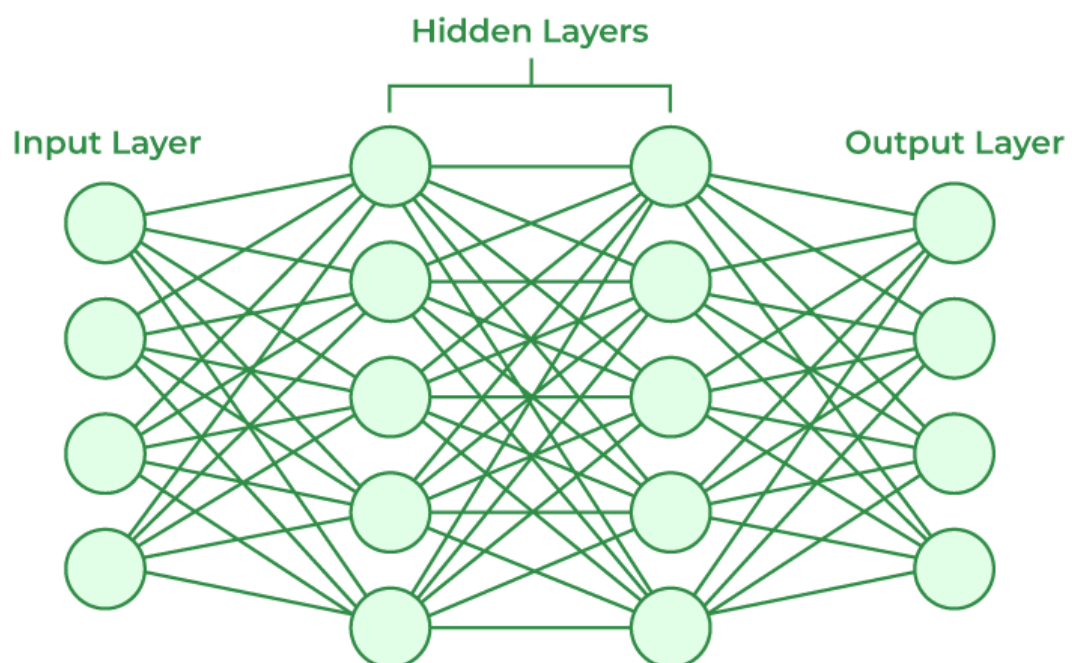
Artificial Neural Network (ANN)

An **Artificial Neural Network (ANN)** is a class of machine learning model inspired (very loosely) by the biological brain. At its core an ANN is a parameterized computational graph that maps inputs to outputs by composing many simple processing units (neurons). Modern ANNs — especially deep ones — are powerful function approximators: given enough capacity and data they can learn complex relationships (classification, regression, sequence prediction, image understanding, etc.).

Basic structure — layers and neurons

An ANN is built from **layers** of simple units called **artificial neurons** (or nodes). The common layer types and the high-level layout:

- **Input layer**
Receives the raw input features (no learning happens here).
- **One or more hidden layers**
Each hidden layer consists of several neurons. The network can be *shallow* (one hidden layer) or *deep* (many hidden layers).
- **Output layer**
Produces the final prediction (class probabilities, regression value, sequence tokens, etc.).



A simple feedforward (fully connected) network with L layers (counting input as layer 0) uses these computations for layer l ($l=1 \dots L-1$):

$$\begin{aligned} z^{(l)} &= W^{(l)} a^{(l-1)} + b^{(l)} && \text{(pre-activation)} \\ a^{(l)} &= \sigma^{(l)}(z^{(l)}) && \text{(activation/output of layer)} \end{aligned}$$

where:

- $a^{(0)}$ is the input vector x ,
- $W^{(l)}$ is the weight matrix for layer l ,
- $b^{(l)}$ is the bias vector,
- $\sigma^{(l)}$ is the activation function (applied elementwise).

Fundamental components

1. Neurons (units)

- Compute a weighted sum of inputs plus a bias, then apply a nonlinearity.
- Single neuron formula (scalar form): $y = \sigma(\sum_i w_i x_i + b)$.

2. Weights and biases (parameters)

- **Weights** determine the strength of connections between units.
- **Biases** shift the activation function so a neuron can activate even when inputs are zero.
- These are the learnable parameters adjusted during training.

3. Activation functions (nonlinearities)

Introduce nonlinearity so networks can approximate complex functions.

- **Sigmoid**: $\sigma(z) = \frac{1}{1+e^{-z}}$ — used historically for outputs; suffers from vanishing gradients for deep nets.
- **Tanh**: zero-centered version of sigmoid; also can saturate.
- **ReLU** (Rectified Linear Unit): $\max(0, z)$ — simple, efficient, alleviates vanishing gradient for many layers.
- **Leaky ReLU / ELU / GELU** — variants to fix “dead ReLU” problem.
- **Softmax**: turns logits into probabilities across classes (used in multi-class classification outputs).

4. **Loss function (objective)**

Measures how well the network's outputs match targets. Examples:

- **Cross-entropy** (classification)
- **Mean Squared Error (MSE)** (regression)
- The training goal is to minimize this loss over the dataset.

5. **Optimizer (training algorithm)**

Uses gradients of the loss w.r.t. parameters to update weights:

- **Gradient descent** (batch / stochastic / mini-batch)
- Adaptive methods: **Adam, RMSProp, Adagrad, SGD + momentum.**
- Learning rate, schedules, and other hyperparameters control the update dynamics.

6. **Backpropagation**

Algorithm to compute gradients efficiently (chain rule applied backward through the network). Backprop + optimizer = training step.

7. **Regularization & training aids**

- **Weight decay (L2), L1:** penalize large weights.
- **Dropout:** randomly zero out units during training to reduce overfitting.
- **Batch Normalization:** normalizes layer inputs to stabilize and speed training.
- **Early stopping:** stop training when validation performance stops improving.

Common layer types / architecture building blocks

Beyond simple dense (fully connected) layers, practical ANNs use specialized layers:

- **Convolutional layers (CNNs)** — for images and local-patterns: use convolution kernels, weight sharing, and pooling.
- **Pooling layers** — spatial downsampling (max, average).
- **Recurrent layers (RNN, LSTM, GRU)** — for sequences/time-series: maintain hidden state across time steps.
- **Attention / Transformer blocks** — for sequences and long-range dependencies (state-of-the-art in language and many other domains).
- **Graph Neural Networks (GNNs)** — for data with graph structure.
- **Residual blocks (ResNet)** — skip connections to ease training of very deep nets.

Each layer type changes the connectivity pattern and number of parameters; choice depends on data structure (images, text, graphs).

How an ANN learns (high-level training loop)

1. **Forward pass:** input $\rightarrow$ network $\rightarrow$ produce predictions.
2. **Compute loss:** compare predictions to targets with chosen loss function.
3. **Backward pass (backpropagation):** compute gradients $\partial L / \partial W$, $\partial L / \partial b$.
4. **Update parameters:** optimizer uses gradients to update weights (e.g., $W \leftarrow W - \eta \nabla_W L$).
5. Repeat for many mini-batches and epochs until performance converges.

Key hyperparameters: learning rate, batch size, number of epochs, architecture size/depth, regularization strengths.

Design considerations, trade-offs & pitfalls

- **Underfitting vs overfitting:** too small network $\rightarrow$ underfit; too large $\rightarrow$ may overfit without enough data/regularization.
- **Vanishing / exploding gradients:** deep networks with certain activations/initializations can experience unstable gradients (mitigated by ReLU, careful initialization, batch norm, residual connections).
- **Initialization matters:** Xavier/Glorot or He initialization help keep signal magnitudes stable.
- **Compute & memory:** deeper/wider models require more GPU memory and compute; batching trades throughput vs gradient noise.
- **Interpretability:** large ANNs can be hard to interpret; various techniques (feature visualizations, attribution) help but are not perfect.

Where ANNs are used (examples)

Image classification, object detection, speech recognition, machine translation, language modeling, time-series forecasting, recommender systems, game playing, medical imaging, and many more.